

Programming Assignment: Unit 2

Hset Paing Htoo

Introduction to Computer Science Program, University of the People

CS 1102-01 Programming 1 AY2026-T5

Instructor : Sirajo Musa (Instructor)

June 28, 2026

Basic Library Management System

1. Introduction

This assignment implements a console-based Library Management System in Java. The program demonstrates core Java control flow constructs covered in the unit readings — specifically while loops, switch statements, and try-catch exception handling — applied to a realistic problem: managing a library catalog.

The system allows users to perform four operations via a menu: add books, borrow books, return books, and exit. Books are stored in a HashMap that maps a lowercase title key to a custom Book object holding author and quantity information.

2. Concepts Applied

2.1 while Loop

The main program loop uses a while loop that continues until the user selects Exit (option 4). According to Eck (2022), a while loop tests its condition before each iteration — making it suitable here because we need the menu to repeat an indefinite number of times, stopping only on a specific user action (Section 3.3).

2.2 do...while Loop vs while Loop

A do...while loop always executes the body at least once before checking the condition, whereas a while loop checks the condition first (Neso Academy, 2020a). In this system, a while loop is the better fit because on start-up it is valid to check whether the system should run at all before

displaying a menu — though in practice a `do...while` could also work since we always want to show the menu at least once.

2.3 try-catch Exception Handling

Wherever user input is parsed as an integer, a try-catch block wraps `Integer.parseInt()`. If the user types non-numeric text, a `NumberFormatException` is thrown and caught, displaying an error message without crashing the program. Eck (2022) explains that exception handling separates the normal logic path from the error path, making programs more robust (Section 3.7). This is also demonstrated in depth by Coding with John (2020), who illustrates how uncaught exceptions terminate a program abruptly.

2.4 switch Statement

After reading the menu choice, a switch statement dispatches to the correct method. This is cleaner than a chain of if-else statements when matching against a small fixed set of integer values.

2.5 HashMap for Storage

The catalog is stored in a `HashMap<String, Book>` where keys are lowercased book titles. Using `toLowerCase()` on the key ensures case-insensitive lookup — so "Clean Code" and "clean code" refer to the same entry.

3. Program Code

The complete source code for LibrarySystem.java is shown below.

```
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

/**
 * LibrarySystem.java
 * A basic library management system.
 */
public class LibrarySystem {

    // Inner class to hold book data
    static class Book {
        String author;
        int quantity;

        Book(String author, int quantity) {
            this.author = author;
            this.quantity = quantity;
        }
    }

    // Library catalog: title (lowercase) -> Book
    static Map<String, Book> catalog = new HashMap<>();

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        boolean running = true;

        System.out.println("=== Welcome to the Library System ===");

        while (running) {
            printMenu();

            int choice = -1;
            try {
                System.out.print("Enter your choice: ");
                choice = Integer.parseInt(scanner.nextLine().trim());
            } catch (NumberFormatException e) {
                System.out.println("[Error] Invalid input. Enter a number
1-4.");
                continue;
            }

            switch (choice) {
                case 1: addBooks(scanner); break;
                case 2: borrowBooks(scanner); break;
                case 3: returnBooks(scanner); break;
```

```

        case 4:
            System.out.println("Goodbye!");
            running = false;
            break;
        default:
            System.out.println("[Error] Choose 1, 2, 3, or 4.");
    }
}
scanner.close();
}

static void printMenu() {
    System.out.println("\n--- Library Menu ---");
    System.out.println("1. Add Books");
    System.out.println("2. Borrow Books");
    System.out.println("3. Return Books");
    System.out.println("4. Exit");
}

static void addBooks(Scanner scanner) {
    System.out.print("Enter book title: ");
    String title = scanner.nextLine().trim();
    if (title.isEmpty()) {
        System.out.println("[Error] Title cannot be empty."); return;
    }

    System.out.print("Enter author name: ");
    String author = scanner.nextLine().trim();
    if (author.isEmpty()) {
        System.out.println("[Error] Author cannot be empty."); return;
    }

    int quantity = 0;
    try {
        System.out.print("Enter quantity to add: ");
        quantity = Integer.parseInt(scanner.nextLine().trim());
        if (quantity <= 0) {
            System.out.println("[Error] Quantity must be positive.");
return;
        }
    } catch (NumberFormatException e) {
        System.out.println("[Error] Invalid quantity."); return;
    }

    String key = title.toLowerCase();
    if (catalog.containsKey(key)) {
        catalog.get(key).quantity += quantity;
        System.out.println("[Success] Updated '" + title + "'. New qty: "
            + catalog.get(key).quantity);
    } else {
        catalog.put(key, new Book(author, quantity));
        System.out.println("[Success] Added '" + title + "' by " +
author
            + ". Qty: " + quantity);
    }
}

```

```

    }

    static void borrowBooks(Scanner scanner) {
        System.out.print("Enter book title: ");
        String title = scanner.nextLine().trim();
        String key = title.toLowerCase();

        if (!catalog.containsKey(key)) {
            System.out.println("[Error] '" + title + "' not found.");
return;
        }

        int requested = 0;
        try {
            System.out.print("Copies to borrow: ");
            requested = Integer.parseInt(scanner.nextLine().trim());
            if (requested <= 0) {
                System.out.println("[Error] Must be positive."); return;
            }
        } catch (NumberFormatException e) {
            System.out.println("[Error] Invalid number."); return;
        }

        Book book = catalog.get(key);
        if (requested > book.quantity) {
            System.out.println("[Error] Not enough copies. Available: " +
book.quantity);
        } else {
            book.quantity -= requested;
            System.out.println("[Success] Borrowed " + requested + "
copy/copies of '"
                + title + "'. Remaining: " + book.quantity);
        }
    }

    static void returnBooks(Scanner scanner) {
        System.out.print("Enter book title: ");
        String title = scanner.nextLine().trim();
        String key = title.toLowerCase();

        if (!catalog.containsKey(key)) {
            System.out.println("[Error] '" + title + "' is not in this
library.");
            return;
        }

        int returning = 0;
        try {
            System.out.print("Copies to return: ");
            returning = Integer.parseInt(scanner.nextLine().trim());
            if (returning <= 0) {
                System.out.println("[Error] Must be positive."); return;
            }
        } catch (NumberFormatException e) {
            System.out.println("[Error] Invalid number."); return;
        }
    }
}

```

```

    }

    Book book = catalog.get(key);
    book.quantity += returning;
    System.out.println("[Success] Returned " + returning + " copy/copies
of '"
        + title + "'. Updated qty: " + book.quantity);
    }
}

```

4. Code Explanation

4.1 Program Entry and Main Loop

The `main()` method instantiates a `Scanner` for user input and sets a boolean flag `running = true`.

The outer `while (running)` loop is the program's heartbeat — it calls `printMenu()`, reads a line, and parses it inside a `try-catch`. If parsing fails, it prints an error and calls `continue`, skipping the switch and restarting the menu (Eck, 2022, Section 3.3).

4.2 `addBooks()`

This method reads three inputs: title, author, and quantity. Each is validated — empty strings and non-positive numbers are rejected immediately with an error message and an early return. If the title already exists in the catalog (checked with `containsKey` using the lowercased title), the quantity is incremented. Otherwise a new `Book` entry is created.

4.3 `borrowBooks()`

After verifying the book exists, this method checks whether the requested number exceeds the available quantity. If so, it prints an error with the available count. If sufficient copies exist, the quantity is decremented and a success message confirms how many remain.

4.4 returnBooks()

Return first confirms the book belongs to the library system — a title not in the catalog cannot be returned. If it is valid, the quantity is incremented. Note that this implementation trusts the return count; a production system would also track which user borrowed how many copies to prevent over-returns.

4.5 Error Handling Strategy

Every integer parse is guarded by try-catch (`NumberFormatException`). Empty string inputs are caught by explicit `isEmpty()` checks. These two patterns together handle the two most common user input mistakes: wrong type and missing value. The program never calls `System.exit()` — it always returns to the menu, keeping the user in control.

5. Program Output

The following output demonstrates all four menu options, including error conditions (insufficient copies, unrecognized return, and non-numeric input).

```
hset-paing@HPH:~/Desktop/code_folder/java_learning/uop-unit-2$ javac LibrarySystem.java
hset-paing@HPH:~/Desktop/code_folder/java_learning/uop-unit-2$ java LibrarySystem
=== Welcome to the Library System ===

--- Library Menu ---
1. Add Books
2. Borrow Books
3. Return Books
4. Exit
Enter your choice: 1
Enter book title: Clean Code
Enter author name: Robert C. Martin
Enter quantity to add: 3
[Success] Added 'Clean Code' by Robert C. Martin. Qty: 3

--- Library Menu ---
1. Add Books
2. Borrow Books
3. Return Books
4. Exit
Enter your choice: 1
Enter book title: Clean Code
Enter author name: Robert C. Martin
Enter quantity to add: 2
[Success] Updated 'Clean Code'. New qty: 5

--- Library Menu ---
1. Add Books
2. Borrow Books
3. Return Books
4. Exit
Enter your choice: 2
Enter book title: Clean Code
Copies to borrow: 4
[Success] Borrowed 4 copy/copies of 'Clean Code'. Remaining: 1

--- Library Menu ---
1. Add Books
2. Borrow Books
3. Return Books
4. Exit
Enter your choice: 2
Enter book title: Clean Code
Copies to borrow: 3
[Error] Not enough copies. Available: 1
```

```
--- Library Menu ---
1. Add Books
2. Borrow Books
3. Return Books
4. Exit
Enter your choice: 3
Enter book title: The Pragmatic Programmer
[Error] 'The Pragmatic Programmer' is not in this library.

--- Library Menu ---
1. Add Books
2. Borrow Books
3. Return Books
4. Exit
Enter your choice: 3
Enter book title: Clean Code
Copies to return: 2
[Success] Returned 2 copy/copies of 'Clean Code'. Updated qty: 3

--- Library Menu ---
1. Add Books
2. Borrow Books
3. Return Books
4. Exit
Enter your choice: abc
[Error] Invalid input. Enter a number 1-4.

--- Library Menu ---
1. Add Books
2. Borrow Books
3. Return Books
4. Exit
Enter your choice: 4
Goodbye!
hset-paing@HPH:~/Desktop/code_folder/java_learning/uop-unit-2$ |
```

6. Discussion

6.1 Design Decisions

Using a static inner class `Book` instead of a two-HashMap approach (one for authors, one for quantities) keeps related data together and makes the code easier to extend — for example, adding a genre or ISBN field requires only modifying the `Book` class.

Case-insensitive key storage (`title.toLowerCase()`) prevents duplicates caused purely by capitalisation differences, which is a common usability issue in catalog systems.

6.2 Limitations and Possible Improvements

Several limitations exist in this V1 implementation:

- No user tracking: the system does not record who borrowed which book or how many, so it cannot enforce return limits.
- No persistence: the catalog is lost when the program exits. A file-backed or database-backed store would fix this.
- No listing command: there is no way to view all books currently in the catalog without adding an option.
- Trust-based returns: a user could return more copies than were ever borrowed, inflating the quantity.

These would all be natural follow-on features in a more complete system.

7. Conclusion

This assignment demonstrated how Java's control flow constructs combine to build a functional interactive application. The while loop sustains the menu, try-catch blocks make input parsing safe, and the switch statement dispatches cleanly between operations. The result is a system that handles both the happy path and common error conditions without crashing — which is the core promise of exception handling as described by Eck (2022, Section 3.7) and Coding with John (2020).

References

Coding with John. (2020, December 31). Exception handling in Java tutorial [Video]. YouTube.

<https://youtube.com/watch?v=1XAfapkBQjk>

Eck, D. J. (2022). Introduction to programming using Java, version 9, JavaFX edition.

<https://math.hws.edu/javanotes/>

Neso Academy. (2020a, March 18). The Do-while loop in Java [Video]. YouTube.

<https://youtube.com/watch?v=2E-UgePiOoo>

Neso Academy. (2020b, March 21). Break and continue keywords in Java [Video]. YouTube.

<https://youtube.com/watch?v=aITdnsUTeek>